NoSQL

CPE 241 Database Management Systems

What is NoSQL

NoSQL = "Not Only SQL"

- A class of databases designed for:
  - Large-scale, distributed data
  - Flexible data structures (no fixed schema)
  - Fast, real-time web applications

Key Characteristics:

- Schema-less
- Horizontally scalable
- High performance for read/write
- Optimized for specific data models (document, key-value, graph, column)

NoSQL is **not a replacement** for relational databases

non-relational data models for **specific workloads** where relational model is not suitable ot inefficient

✅ Often used alongside **Node.js & Express** in modern web apps

Why Traditional SQL Might Not Be Suitable

SQL databases (e.g., MySQL, PostgreSQL):

- Rigid schema → hard to change fields during rapid development
- Complex joins → performance drops at scale
- Vertical scaling → upgrading server limits

When SQL more complex and expensive:

- Real-time systems (e.g., chat apps)
- Big data and IoT
- Rapidly evolving data models (startups, MVPs)
- Massive concurrent users

SQL can solve these problems, but often with higher **complexity and cost**

What Can NoSQL Do?

NoSQL databases are ideal for:

- **User profiles** (flexible, nested data)
- **Product catalogs** (with dynamic attributes)
- **Real-time analytics**
- **Caching** (e.g., Redis for fast lookups)
- **Social networks** (graph databases)

Examples:

- **MongoDB** → Document DB for web apps
- **Firebase** → Real-time DB for mobile/web
- **Redis** → Caching and session storage
- **Neo4j** → Relationships in social apps

Application checks relationship and integrity
NoSQL lets data structure follow the product, not the schema.
SQL excels relational queries and integrity

SQL vs NoSQL

Feature	SQL	NoSQL
Full Name	Structured Query Language	Not Only SQL
Database Model	Relational	Non-relational
Data Format	Tables (rows & columns)	Documents, key-value, graph, column
Schema	Fixed (DB-enforced)	Flexible (app-enforced)
Scalability	Vertical (horizontal with added design)	Horizontal (built-in)
Query Interface	SQL	Database-specific APIs
Best Suited For	Transactions, analytics, strong integrity	Flexible data, scaling, app-centric models

In practice, “SQL vs NoSQL” usually means **relational vs non-relational** databases.

NoSQL Data Models & Popular Databases

Data Model	Structure	Popular Databases	Best For
🗒️ **Document**	JSON / BSON documents	MongoDB, CouchDB, Firebase Firestore	Content management, user profiles, catalogs
🔑 **Key-Value**	Key → Value	Redis, Amazon DynamoDB, Riak	Caching, session storage, real-time counters
📊 **Column-Family**	Rows grouped by column family	Apache Cassandra, HBase, ScyllaDB	Time-series, analytics, event logging
🔗 **Graph**	Nodes + Edges	Neo4j, ArangoDB, Amazon Neptune	Social networks, recommendation engines

Tech Stacks

Frontend: React / Vue / HTML + JS
Backend: Node.js + Express
Database: MongoDB (via Mongoose)

* This is an RDBMS course. You only need to be aware that non-RDBMS databases exist in your toolbox — no need to master them now.

** Notice that most of the stack remains the same; we simply switch MongoDB in place of PostgreSQL.

*** In real-world systems, it is common to use both SQL and NoSQL databases in the same application.

Getting Started with MongoDB

What is MongoDB?

- A **document-based NoSQL database**
- Stores data in flexible, JSON-like **BSON** documents
- Great for web apps, fast prototyping, and handling varied data

Installing MongoDB

Local

MongoDB Community Edition + Compass

Cloud

MongoDB Atlas

MongoDB Structure

Database is a **logical container** for collections. Each MongoDB instance can hold **multiple databases**

Collection is a **group of documents** (like a table in SQL). Schema-less → documents can vary in structure

Document is the **basic unit of data** in MongoDB. Stored in **BSON** (Binary JSON). Flexible structure → supports nesting, arrays, and mixed types

```
{
  "name": "Alice",
  "age": 21,
  "skills": ["Python", "MongoDB"],
  "address": {
    "city": "Bangkok",
    "zip": "10110"
  }
}
```

Each field = **key-value pair**
Values can be:

- String, Number, Boolean
- Array, Object (nested)
- Null, Date, ObjectId, etc.

MongoDB Core Operations

You can do everything in GUI interface call “Compass”

📁 Database Operations

- `show dbs` – List all databases
- `use myDatabase` – Switch to or create a database
- `db` – Show current database

📦 Collection Operations

- `db.createCollection("students")` – Create a new collection
- `show collections` – List all collections in the current database
- `db.students.drop()` – Delete a collection

Document (CRUD) Operations

Create:
db.students.insertOne({ name: "Alice", age: 21 })
db.students.insertMany([{...}, {...}])

Read:
db.students.find()
db.students.find({ age: { $gt: 20 } })

Update:
db.students.updateOne({ name: "Alice" }, { $set: { age: 22 } })
db.students.updateMany({}, { $inc: { age: 1 } })

Delete:
db.students.deleteOne({ name: "Alice" })
db.students.deleteMany({ age: { $lt: 18 } })

Node.js + Express with MongoDB

- 🟢 **Node.js** lets you build fast, scalable server-side apps with JavaScript
- 🚏 **Express.js** is a lightweight web framework for Node.js
- 🍃 Together with MongoDB, they form the **MERN stack** (Mongo, Express, React, Node)

Why this combo?

- Same language: JavaScript across frontend & backend
- Non-blocking I/O: handles many users efficiently
- Works well with JSON (like MongoDB documents)

What Will Our App Do?

We'll build a **simple web app** with:

- 📝 A form (HTML) to submit user data (e.g., name, email)
- ⚙️ An Express backend to handle the request
- 📦 MongoDB to store submitted data
- ✅ A confirmation message to the user

The Full Request Lifecycle :How It All Connects

User submits form →

Frontend (HTML + JS) →

Backend (Express Route) →

MongoDB (via Mongoose) →

Response sent back to user

Tech Stack Overview

We're building a **full-stack web app** using:

🧠 Everything communicates using **HTTP** and data is passed in **JSON** format.

Layer	Technology	Role
Frontend	HTML + JavaScript	UI for user interaction
Backend	Node.js + Express	Handles HTTP requests/responses
Database	MongoDB + Mongoose	Stores and retrieves user data

What is Express.js?

- A **minimalist web framework** for Node.js
- Lets you define **routes** and **middleware** easily
- Used to handle:
    - HTTP Methods: GET, POST, PUT, DELETE
    - APIs and endpoints
    - Request/response processing

Express Routing – The Basics

In Express, you define **routes** to handle specific requests:

- `GET /` → Show homepage
- `POST /submit` → Receive form data
- `GET /users` → Return user data

Each route consists of:

METHOD + PATH → HANDLER FUNCTION

What is Middleware in Express?

Middleware = functions that run **before** the final request handler.

They can:

- Modify request or response objects
- Run code (e.g., logging, auth)
- End the request or pass it along

📦 Examples:

- `express.json()` – Parses JSON body
- `express.static()` – Serves static files (like HTML/CSS)
- Custom middleware for logging or validation

JSON – The Language of Data

Explain why JSON is the common format:

- MongoDB stores documents as JSON-like **BSON**
- Express uses JSON to handle API requests and responses
- JavaScript (frontend/backend) natively supports JSON

🔑 Basic Structure: Key-Value Pairs

Keys (also called field names) are **strings in double quotes**
Values can be:

- String (`"Alice"`)
- Number (`21`)
- Boolean (`true`, `false`)
- Null (`null`)
- Array (`[1, 2, 3]`)
- Object (`{ "city": "Bangkok" }`)

```
{
  "name": "Alice",
  "age": 21,
  "isStudent": true
}
```

⚠️ JSON Rules (Syntax Musts)

- Strings must use **double quotes** `"  "` (not single quotes)
- No trailing commas after the last item
- JSON is **case-sensitive**
- Only supports a limited set of data types

Mongoose – MongoDB(Driver) for Node.js

Mongoose is an **ODM (Object Data Modeling)** library for MongoDB and Node.js.

- Adds structure to MongoDB's schema-less documents
- Makes working with MongoDB **easier and safer**
- Provides built-in tools for:
    - Data modeling with **schemas**
    - CRUD operations
    - Validation and type checking
    - Query building

Suggested Self-Study Activities

quick preview (no detail yet):

1. Install MongoDB & access via shell
2. Create DB, collection, and documents
3. Build CRUD API with Express
4. Connect HTML frontend to backend
5. (Optional) Explore Compass

Wrap-Up — Key Takeaways

- NoSQL = flexible, scalable, and great for modern apps
- MongoDB stores data as documents, not tables
- Node.js + Express = fast and JS-friendly backend
- Mongoose = structured way to work with MongoDB
- JSON = glue between all layers of the stack